Stage 3. Gateways and data

An entry backup, gateways, scheduled tasks, acceptance data and handheld routes

PREPARED BY Energy Hippo implementation team
STATUS Ready to enter

Stage 3 is where Northlake stops being an empty hierarchy and starts receiving measurements. It takes a database checkpoint before anything changes, configures the sixteen gateway profiles that cover every acquisition method the product ships (file delivery, device polling, HTTP weather services, ODBC query, a synthetic generator, and two handheld upload formats) and gives Ridgeline Wind's points their sources, creates reusable tasks with per-gateway schedules, proves the AcquiSuite path end to end against exact expected numbers, and sets up the handheld meter reading routes that the two handheld gateways depend on. When its checks are green, Northlake has real interval data in it and Stage 4 can build rates against real consumption.

Before you start

- Stage 2's checks are green: every point this stage attaches to a gateway already exists, with its time zone and interval set.
- Database Administrator, the web site and the Workers service (EEMSuite Task Service (NorthLake)) are all running. Nothing in this stage runs without Workers; a gateway that only produces an intermediate file or a task log entry has not proven anything.
- You have sysadmin access to the SQL Server instance for step 1's entry backup, and file system access to `C:\ProgramData\EEM Suite\NorthLake\` for the gateway folders step 2 creates.
- Enter everything through the applications. A screen that cannot take a value this stage needs is a finding, recorded at the end of this chapter, never worked around with SQL.

Step 1. Database checkpoint

Where: SQL Server, `BACKUP DATABASE` (sqlcmd or Management Studio). This is not an EEM editor; it is the way back if gateway entry goes wrong.

Enter: 01-checkpoint.yaml, one backup, `stage3_entry`, taken before step 2. Northlake's was taken on October 5, 2026.

No second backup in this stage (decided November 5, 2026). Stage 3 takes no configured-empty checkpoint. Northlake is backed up later, once it works end to end. The acceptance runs in step 4 therefore run on the live database, in an order that needs no restore, and the data they load stays.

What a restore does not undo: the gateway Input/Working/Backup/Error folders and the GatewayOutput and GatewayCaches folders under `C:\ProgramData\EEM Suite\NorthLake\`. If a backup is ever restored, stop the Task Service (EEMSuite Task Service (Northlake)) and the Northlake site in IIS first, empty those folders by hand after it, and start both again.

Record: the backup file path and the date taken.

Check: none; the backup is recorded in `01-checkpoint.yaml`.

Step 2. Gateway types, then gateways

Where: Configuration Tool, two screens in this order. **Gateway Types** defines each type: its name, and for each node and point parameter the column it is stored in (Node or Point Parameter 1-15), its label, its type (String, Boolean, Integer or Decimal) and any predefined values. **Gateway Configuration** then creates each gateway on its type: a gateway's nodes and points show exactly the parameters its type defines, so a gateway can only be entered once its type is right.

Enter: 02-gateways.yaml, in two parts.

- **gatewayTypes**, nine types, first. Seven are seeded and need checking against the recipe: AcquiSuite Gateway, BACnet Gateway, Modbus Gateway, Spinwave Gateway, FIG Gateway, MDEF (both MV90 formats) and ODBC Gateway. Two are new: Weather Gateway, which NOAA and Aeris Weather share, and Fake Gateway. Each parameter's Reads says what its task component does with the value, so the labels match the code. Add what a seeded type lacks (finding 21): BACnet's node parameter BBMD UDP Port, and FIG's point parameters Channel, UOM, Multiplier and OffsetTimeMinutes (the CMEP point builds its alias from the first two, and Ridgeline's SCADA depends on the last).
- **records**, 15 gateway profiles, after the types: 12 acquisition gateways and the three Neptune and MVRS handheld gateways. HMR Events Publish runs inside the MVRS workflow and has no gateway row. One of the 12 is Ridgeline Wind's SCADA gateway, and two existing gateways each gain a Ridgeline node (see Ridgeline Wind below). The handheld gateways take the parameterless Handheld Meter Reading Gateway type and no nodes; they are created in step 5 (finding 78).

What the upgrade corrects: `RepairGatewayParameterMetadata.sql` makes every enable flag a Boolean and the numeric parameters Integer, and makes BACnet's Object Type and Property Integer; as seeded they are Booleans with six values and one, and Gateway Types will not save the BACnet type until that is fixed. It also deletes the eight legacy types no 7.0 component reads, wherever no gateway uses them; `otherGatewayTypes` lists them, and the two product types Northlake leaves alone (Aggregation Gateway and My Imports Gateway). On a build without that script, make the same corrections by hand.

Order inside the editor: on Gateway Types, save each type before leaving it. On Gateway Configuration, create the gateway and save before adding nodes; save a node before adding its points (an unsaved node has no id for a point to reference). Leave IsEnabled blank, which means enabled, and leave blank every parameter marked not read by 7.0.

Current editor: installed version 7.0.0.4090 has no Delimiter or legacy display-order (`GWParamPos`) input, so the recipe and checker do not require those stored values. The Gateway picker beside Description only shows a component's suggested fields; its selection is not saved. Add the fields in the recipe, not every suggestion. Predefined values must be nonblank: BACnet's Array Index choice is `-1` / `Typically -1`, while the point's actual value can still stay blank.

Where this is actually stored, since there is no table named "Gateway": the gateway is a row in `ObjectRegistry`, a node is a row in `GwNodes` with fifteen free-text parameter columns, and a point is the same `PointDef_TBL` row Stage 2 created, now pointed at a node with its own fifteen free-text columns filled in. `GWVendor`, `GWVendorParam` and `GWVendorParamVal` hold the types, and the worker component reads each value back by its label (for example `AcquiSuiteGatewayComponent` reads a node's `Serial Number` and a point's `DeviceID` and `Row Number`).

Neptune and MVRS are the exception: the gateway identifies the run, but the current `HMR_PointNode_Get` procedure loads all handheld configurations without filtering them by gateway. The returned file selects a physical meter by its hierarchy ID and a channel by Text Prompt. The meter's prompt, constant, and register/usage pairing live in the HMR tables configured through the web. Separate gateway folders are not a permission or meter-selection boundary. The gateway still has to exist so a schedule can select it, and Configuration Tool will not save one without a type, so the three handheld gateways share an empty Handheld Meter Reading Gateway type that the components never read. See step 5.

Large point sets: the fixed-network submeter gateway carries 128 points, one per apartment plus the two house meters, the laundry and the EV chargers. The Points grid's bulk edit sets the shared values (Channel, UOM, Multiplier) across every selected row at once; only the per-unit alias is typed per row. 02-gateways.yaml gives the naming pattern and the count, not 128 individual lines. The Cedar Row water submeters are handheld meters: they go on no gateway and are set up in step 5.

Node flags and disabled records (added 2026-11-10). Three things on the node and point pages that earlier versions of this step left at their defaults:

- **Enable Data Gaps** is on for NOAA-KSAC, NOAA-KSMF and NTP-BACNET-PLANT-HEADERS, the feeds that are polled live. The Stage 9 task Run DataGap Calculations (`hmDataGap_Exec`) only looks at points on a node with the flag on, so before this it had nothing to do. It scans the last 100 days (the `hmDaysBack` system constant) and, the first time it sees a point, flags every hourly interval from the start of that window to the point's latest reading as missing. Expect thousands of gaps per point on its first run after a live poll, and none for any node whose data is only from 2024: that is why the AcquiSuite and MDEF nodes are not on this list.
- **Enable Late Intervals** is on for ASQ-VED-STUDENT, MV90-MDEF-VED-SCIENCE and FIG-CEDAR-ROW-A-UNITS, the feeds whose data arrives by file, and each of the three needs a **Next Batch Method** other than Disabled: the late window ends at the method's next batch time, and the Late Intervals report and Gateway Status report join on the method. The list is database data, not source, so pick the entry that matches a daily delivery and write its name into 02-gateways.yaml. Because those three feeds stopped delivering in 2024, the late scan flags the last 100 days of every point on them and the report shows each node at 0 percent posting, Fail: the right answer for a feed that has gone quiet, and one that clears only when a current file is delivered.
- **One node and one point are disabled on purpose.** ASQ-VED-LAKEVIEW has IsEnabled 0 (the Lakeview Hall logger was pulled in the renovation) and the Lakeview Hall laundry submeter on the FIG campus node has Disabled 1 (the laundry room closed). The AcquiSuite gateway skips the node and counts its two points as skipped; the FIG gateway skips the point and counts it in Skipped Point Count. FIG reads no node IsEnabled, so only its points can be disabled.

Record: the gateway id, each node id, and each point's parameter values once saved.

Check: `gateway_types`, `gateway_types_retired`, `gateway_types_unlisted`, `gateway_points_bound`, `ridgeline_nodes_bound`, `node_scheduling_flags`.

Step 3. Workflows and tasks

Switch the gateway kinds on first. In Profile Manager, System Configuration, set the key `Gateway/Objects/Enablement` (JSON) to the value in `gatewayEnablement` at the top of 03-workflows-and-tasks.yaml. Every gateway component checks its kind there before anything else, and every kind defaults to off, so until it is set each gateway task fails at once with "disabled by configuration key".

Where: Configuration Tool > Tasks > Northlake Gateways defines the reusable task and its defaults. Web > Tasks > Library creates its schedules and supplies their parameter values. The schedule drawer sets Schedule Kind (Runs once, Interval or Cron), and for Interval, how often. Keep every schedule disabled until step 4.

Enter: 03-workflows-and-tasks.yaml, **11 reusable tasks and 16 schedules**. Reuse the ten built-in workflows and create only the custom two-node `FakeGateway → IntervalDataPublish` workflow. MVRS already publishes HMR events in its built-in workflow; it needs no separate HMR task.

Leave task-default GatewayId, RootDirectory and other per-schedule inputs blank, even when the workflow marks them Required. Required means that an effective value must exist when it runs; the schedule can supply it. Keep parameter fields visible, Can schedule enabled and Single schedule only disabled. Each task gets one BVirtualPC execution target with Run as blank. Keep existing targets; their hidden legacy gateway metadata does not override the scheduled GatewayId that the collector uses to load its gateway configuration.

Use one MV90 task for MDEF and MV9, one NOAA task for current and history, one FIG task for all three FIG profiles, and one Neptune task for both water routes. Each named schedule supplies its own GatewayId and RootDirectory, plus Format, Operation or other overrides as listed in the YAML. Ridgeline SCADA is the ten-minute schedule on the shared FIG task. Set only RootDirectory for a normal file workspace; Input, Working, Archive, Error and Output resolve beneath it.

The floor that matters: whatever interval is typed, the server enforces at least 300 seconds for any task. Every schedule in this stage respects that already; only the Fake Gateway smoke test runs once on demand instead.

Two workflows the catalog lists that this stage does not use: OdbcFileDataGateway is a misnamed DB-to-file exporter, not an acquisition gateway, and Northlake has no outbound feed that fits it. BillImporter belongs to the billing stage. Neither gets a task here.

Record: each reusable task id, the workflow it runs, and each schedule's name, id and overrides.

Check: the `BEGIN WORKFLOW CHECKS` batch in checks.sql. It checks task defaults and effective schedule values separately, including blank required defaults, visible fields, target bindings and duplicate per-profile tasks. See the stage README for the deferred live migration.

Step 4. Acceptance runs

Where: the gateway's Input folder (file delivery), Task Manager's Run now, and Reports > Use Analysis (Time Interval for counts and timestamps, Summary for the kWh sum and kW maximum) to read back what was actually stored.

Enter: 04-acceptance-runs.yaml. The AcquiSuite case is four runs against the customer's own expected numbers, in one sequence on the live database with no restore: a file with a missing block, a backfill that closes it, the complete day over the filled day, and a replay of that same file. The remaining 31 format cases (five of them Ridgeline's, below, and one the Central Plant's August 2024 bill period) each carry a Status block: whether a sample exists, what would produce it, the expected result and the check. Only AcquiSuite has sample files in the repository. Most of the others are made by the `services/sources` producer, which delivers into a gateway's Input folder with `python -m services.sources deliver`; the BACnet, historian and handheld cases have their own services; Modbus has none yet; NOAA is public data with no fixed value. A check that reads NOT LOADED means the data is not in the database yet, not that a case failed.

The numbers that must match exactly: the gap file, run first, lands 88 readings per point and 1,780 kWh, missing 12:00 through 13:45 Pacific. The backfill file supplies exactly those eight intervals, 240 kWh, and brings the day to the complete local day (2024-01-15 Pacific, UTC 2024-01-15 08:00 through 2024-01-16 07:45): 96

readings per point, 2,020 kWh summed, 120 kW at the peak. The complete-day file and then its replay land on that full day and must leave 96 unique timestamps and 2,020 kWh; record how the product reports them (update in place, or something else). Sum kWh; take the maximum of kW; never sum demand.

The AcquiSuite files stamp interval starts, on purpose (finding 73). They are the customer who assumes start-of-interval stamps. EEM keeps the stamp as given and reads every stamp as the end of its interval, and no AcquiSuite setting can say otherwise, so the file's first row, midnight Pacific, is January 14's last interval to EEM. The stored rows are complete (96 and 2,020 kWh on the file's own day), but EEM's January 15, which is what the reports show, holds 95 readings and 2,010 kWh. Record both; that is the finding, not a failed load.

A task log success is not acceptance. Acceptance is those numbers rendering on the Time Interval and Summary reports for the affected points.

Record: the workflow run id and the observed numbers for every case run.

The Central Plant case (added 2026-11-10). The main meter is now rated VED-TOU-PRI, so its interval points need the bill period of `ved_central_plant_2024_09` behind them. The case is `mdef_central_plant_2024_09`: gateway Northlake VED MDEF Interval Import, node MV90-MDEF-VED-PLANT, points Delivered kWh 15m and Demand kW 15m (recorder VEDPLANT01, channels 1 and 2), local stamps after 2024-08-05 00:00 up to and including 2024-09-05 00:00, which is 2,976 readings per point. Cut by the Stage 4 grids, the kWh must sum to 412,800 peak (528 intervals), 368,400 part-peak (704) and 731,200 off-peak (1,744) kWh, 1,512,400 in all, with a 3,240 kW maximum; re-running the Stage 4 test with Load from Point on these points must then give the bill's $303,951.62. Run `central_plant_tou_period_totals` to see each figure beside its expected value.

Check: `acquisuite_complete_day`, `replay_no_duplicates`, `gap_then_backfill`, `task_runs`, and for the other format cases `mdef_valley_electric_month`, `central_plant_tou_period_totals`, `mv9_helios_rules`, `bacnet_history_tie_out`, `noaa_observation_sanity`, `noaa_history_scope`, `spinwave_days`, `fig_cedar_row_water_days`, `fixed_network_days`, `feed_archive_intervals` and `fake_smoke_day`. Each prints Expected, Actual and a Status. `replay_no_duplicates` now counts every archived row of the two Student Center points (it used to test the archive's primary key, which cannot fail).

Step 5. Handheld meter reading routes

Where: Web > System Manager > Handheld Meter Setup (`#/HandheldMeterSetup`), which takes over the HMR Configuration menu entry before this step. Its **More** menu holds **Read types** and **Paste from Excel**; its **Route file** action opens Handheld Meter Reading Export (`#/HandheldMeterReadingExport`).

Enter: 05-handheld-routes.yaml, **192 meters and 204 registers on ten routes**. The five set up first:

- the Cedar Row B water master (Neptune);
- the Student Center gas master (MVRS);
- the two Cedar Row A and B apartment and house water routes (126 meters, Neptune);
- the sewer-credit deduct route (4 meters, Neptune).

The five added November 7 are there to learn the handheld settings, one reason each:

Route	Meters	Registers	What it teaches
`02/CRCWTR`	50	50	Cedar Row C as its installer keyed it: two meter batches (ccf upstairs, gallons downstairs), a read order that zig-zags between the two meter banks, unit 412 on cycle 20, the laundry meter disabled, meter text copied from Cedar Row B
`04/IRRIG`	4	4	the easy one: irrigation check reads in ccf, in season

Route	Meters	Registers	What it teaches
01/CHECK	2	2	a meter constant that applies a meter multiplier (x40 CT ratio, x3.0367 pressure factor)
03/SWGR	1	10	one meter with ten registers: kWh, three time-of-use kWh, kVArh and kVAh through their Deltas, and kW, on-peak kW, kVA and power factor as demand reads
05/GEN	3	6	two registers per meter: run hours and diesel gallons

Cedar Row C's mistakes are the expected configuration here; the Stage 12 scenario fixes them with bulk edit. The meters and points come from Stage 2, which added Cedar Row C, the generators and the switchgear on November 7. Every register is generated into 05-handheld-meters.csv, ready to paste.

Read types: More > **Read types**: add the nine Northlake entries in the YAML and save them together. The water, gas, kWh and kW formats follow the HMR research (codes 65, 88, 10 and 13); hours, fuel and power factor take new Northlake codes 70, 71 and 72. They are Northlake identities, not Massport IDs.

Meters and registers: choose Northlake University, Cedar Row Apartments and Northlake Thermal Plant and show all meters. **More** > **Paste from Excel**: open the CSV in a text editor, not Excel (Excel turns cycle 01 into 1 and tamper code 00 into 0), copy all of it, paste it and save. The page matches each row to its **physical meter** by meter number; a meter with several registers has one row for each, told apart by its prompt. A row fills the meter's cycle, route, sequence, enabled flag and meter text, and the register's read type, prompt, constant, channel, tamper code, high and low warning, Reading Point and Consumption Point: a register's Reading Point is its Stage 2 register and its Consumption Point the generated Delta; a demand read leaves Reading Point empty and names its Analog point. The save is all or nothing and marks any rejected row. Meter text avoids `;` `,` `( ) / ' "` and the like, which the page saves as `_`.

Starting readings: two passes, Cedar Row C first. Choose only Cedar Row C, set the date to **September 1, 2025**, open 05-handheld-starting-readings-2025-09-01.txt in a text editor, copy its 50 lines, paste them into the first row's Starting reading and save. Then choose the three companies, set **January 1, 2024** and paste the 150 lines of 05-handheld-starting-readings-2024-01-01.txt. Each value becomes its register's first reading at the start of that day, with no usage. Each file is in the grid's order, so it lines up only when the grid lists exactly 50, then 150, registers with no starting reading. Demand reads take none and are not listed.

Handheld gateways: in Configuration Tool, add the **Handheld Meter Reading Gateway** type on Gateway Types with no node or point parameters, and save it. Then, on Gateway Configuration, add the three gateways with that type and nothing else, no nodes and no points: **Northlake Neptune Water Upload**, **Northlake MVRS Gas Upload** and **Cedar Row Submeter Water Route**. Configuration Tool requires a type; Neptune and MVRS never read it. They still work node by node, but `HMR_PointNode_Get` supplies one node of its own and fills it with the meters set up above, so the gateway needs none. Record the three gateway IDs, then save their disabled schedules on the Neptune and MVRS tasks (03-workflows-and-tasks.yaml).

`HMR_Reading` is configuration, not observation history: entering it adds no readings, and the starting readings are the first. The Step 4 HMR pack checks the masters' starting readings, then exercises normal and zero usage, file versus database previous values, replay, MVRS skips and late actuals, logs/text, a batch of 126 submeters and invalid input. With no restore point in Stage 3, its probe readings stay in the meters' history. Direct demand, multipliers and rollover change settings temporarily and leave synthetic values, so they wait until a backup exists to restore afterward. The direct-demand case uses an existing point without changing the permanent Stage 2 inventory.

Each route added November 7 has its own case in the pack, and none of them changes a setting:

- **CHECK:** the two meter constants on the Deltas (375 x 40 = 15,000 kWh, 800 x 3.0367 = 2,429.36 ccf).

- **SWGR:** the switchgear's ten readings from one meter record, six through Deltas and four stored as read. They agree with each other: the time-of-use kWh add up to the total, and kWh over kVAh matches the power factor read.
- **GEN:** two registers per meter, told apart only by their prompts.
- **IRRIG:** the season's first irrigation read, covering January to May.
- **CRCWTR:** Cedar Row C as built. 48 meters are read; unit 412 and the laundry meter keep only their starting readings.

Export after the starting readings to verify the outgoing file carries them as previous values. A filtered grid does not limit the download. Export disables cycle 99 assignments, so none of these routes use that cycle. Save the actual file as evidence; an export is not an inbound upload. Handheld source files still need generating from the installed meter IDs.

Northlake exercises ordinary HMR data and logs. Massport's skipped-read estimation/approval, LOTO and custom report editing are separate customer extensions and are not required here.

The agreed handheld testing environment will add an optional service at localhost:18082 with separate saved sessions, real export parsing and two native return formats. The first two-route demonstration uses controlled whole-unit values. Retained canonical history requires the explicit raw-scale, conversion, export-width and water-conservation gate; the current source precision cannot simply be copied into the native register fields. Stage 9 reuses these assignments for ongoing reading, delivery, review, replay and corrections.

Record: the three handheld gateway IDs, physical meter ID, HMR meter and reading IDs, register and usage point IDs, export file, input hashes, task run IDs, persisted values, and applicable HMR log entries.

Check: the `handheld_configuration` section of `checks.sql` compares every expected route, register, format and register/Delta or demand point and reports unexpected assignments and registers. The nine read types and 204 registers, **213 checks**, must pass before the step 4 handheld pack. Configuration checks do not prove the later import or export exercises passed.

Step 6. Baselines

Where: Database Administrator, Nodes, Point nodes: the Baseline group on the baseline point's own Definitions page.

Enter: 06-baselines.yaml: the Baseline group on the 3 baseline point shells Stage 2 created. Set Baseline Algorithm, tick Display baseline, and set Baseline For to the measured point each one baselines.

When: after Step 2. Baseline For lists only sibling points with a commodity unit that already have a gateway (or a registered aggregate), so it is empty until Step 2 has bound the measured points; this is why the step moved here from Stage 2.

Two algorithms, one of each kind: Rolling Day Type Forecast (a forecast that creates data) on the two Science Center baselines, and Average of Top 5 Days (a curtailment baseline) on the Student Center demand baseline, which bids into Valley Electric District demand response.

Seven seeded algorithms are not used (Average of 3 Days in 7, Yesterday, Last 5 Days, Last 10 Days, Other Baseline, Other Forecast and EnergyAI Baseline). A baseline needs a Stage 2 point shell and enough history to give a known result; the recipe has neither, since the Student Center demand point holds one loaded day and Average of Top 5 Days already needs five.

Record: nothing new; these are edits to points recorded in Stage 2.

Check: `baselines`.

Ridgeline Wind

Added 2026-10-01 (Ridgeline Wind). The wind company the Thermal Plant bought in July 2024 reaches Stage 3 through three gateways, and none of them is new technology: one new FIG gateway for the SCADA files, and one new node each on the VED MDEF gateway and the NOAA gateway. Together they give all 72 of Stage 2's Ridgeline points a source.

What to enter: in step 2, Ridgeline SCADA File Imports on the FIG Gateway vendor, with FIG-RIDGELINE-TURBINES (60 points, six on each of the ten turbines) and FIG-RIDGELINE-MET-MAST (6 points); MV90-MDEF-VED-RIDGELINE (3 points, recorder VEDRIDGE01) on the VED MDEF gateway; and NOAA-KSUU on the NOAA gateway. In step 3 the SCADA gateway gets a ten-minute schedule on the shared FIG task. The other two nodes use the existing MDEF and NOAA schedules.

The SCADA clock never changes. Ridgeline's SCADA server keeps Pacific Standard Time all year, and EEM has no fixed UTC-08:00 time zone, so the file is read through the FIG gateway's UTC plus offset. The 66 SCADA points keep their Pacific time zone, as Stage 2 left them, and each carries `OffsetTimeMinutes` 480: the FIG gateway adds eight hours to every stamp in the file, which turns the SCADA's wall clock into the UTC instant in January and in July alike. The Ridgeline task's Timestamp Type is `utc`, not the default `local`, so that shifted stamp is read as UTC and EEM converts it into Pacific for the point's stored local time, which comes out right all year. Under `local` the shifted stamp would be read as Pacific wall-clock time and stored 7 or 8 hours late. The FIG Gateway vendor needs its `OffsetTimeMinutes` and `Multiplier` labels added first (step 2, finding 21); without the offset every stamp lands eight hours early, with no error. Do not read the file as GMT instead: that zone (number 23) is stored under the Windows name GMT Standard Time, which is London time with summer time (finding 48), and FIG offers no standard-time stamp type that would need no offset (finding 49). Step 4 checks the recipe on a July day, when the SCADA clock and the Pacific clock are an hour apart.

KSUU wind is in knots (finding 45). The NOAA gateway maps the field name `wspdm` to the observation's `wind_kt`, which is knots, although `wspdm` means km/h in the naming it inherits; nothing on screen says so. KSUU's wind point is set up in knots, Northlake's own unit, and a point built from the field name alone would read 1.852 times too low. The gateway has no wind-direction field, so KSUU has no direction point.

The statement month. Valley Electric bills run from the 5th to the 4th, so the generator's first bill covers 2024-07-01 to 2024-08-04 and its winter bill 2025-01-05 to 2025-02-04, while Ridgeline's own transfer statement is a calendar month. The statement slices on Pacific civil months, which under this recipe are the points' stored local time, and Bill Generation cuts the month the same way.

Every stamp is an interval end. EEM's rate engine, Bill Generation and interval reports read a stored stamp as the end of its interval: a day is the stamps after its midnight up to and including the next. The SCADA file is written that way, each row stamped at the end of its ten minutes, and the MDEF gateway stamps each reading at the end of its fifteen. So a SCADA day runs from 00:10 to the next 00:00, and the full history starts with the row stamped 2024-06-30 23:10 on the SCADA clock, the first ten minutes of the Pacific July. The checks cut every Ridgeline window the same way.

Acceptance: five cases at the end of 04-acceptance-runs.yaml, none with a generated sample yet. A July 2024 SCADA day must land 144 readings per point at the right UTC (08:10 through 08:00 the next morning, because the SCADA's 00:00 is 08:00 UTC even in summer) with stored local stamps that read Pacific civil time, 01:10 daylight time first, which is what verifies the UTC-plus-offset workaround (finding 49). The WTG-07 gearbox outage, 2025-02-11 through 2025-04-22, must arrive as rows with state 2 (Fault) and zero energy, not as a gap: a fault is lost production, and the controller keeps reporting possible power. KSUU wind must arrive in knots,

checked against the met mast's metres per second times 1.943844. The interconnection meter's three MDEF channels must land on their own points and sum, over Valley Electric's 5th-to-4th bill periods, to the generator bills' metered quantities. And the four Ridgeline aggregates must calculate once their members have data, which, like every aggregate, waits on product finding 14.

Record: the run id and the observed numbers for each case, and the stored local stamps for the July day.

Check: `ridgeline_nodes_bound`, `ridgeline_scada_day`, `ridgeline_scada_day_short_points`, `ridgeline_wtg07_outage`, `ksuu_wind_knots`, `ridgeline_mdef_channels`, `ridgeline_aggregates`, `ridgeline_period_totals`.

Stage 3 is done when

- All 15 gateway profiles exist: the 12 acquisition gateways with their nodes and points attached (Ridgeline's 72 points included: its SCADA gateway and one node each on the MDEF and NOAA gateways) and the three handheld gateways on Handheld Meter Reading Gateway with no nodes. Every gateway type matches 02-gateways.yaml (`gateway_types` and `gateway_type_assignments` empty), and the legacy types are gone.
- The gateway kinds the tasks use are switched on in `Gateway/Objects/Enablement`.
- 11 reusable tasks serve 16 named schedules, each with the correct gateway and run-specific parameters, with no interval below 300 seconds.
- The AcquiSuite complete-day, replay, gap and backfill cases show the exact expected numbers on the Time Interval and Summary reports.
- Every remaining format case, Ridgeline's five and the Central Plant's included, is recorded as either run, or open work waiting on its sample file or service, and its check reads PASS (or NOT LOADED while it waits).
- `node_scheduling_flags` reads PASS on every row: the data-gap and late-interval flags, the disabled Lakeview Hall node and laundry point.
- The July 2024 Ridgeline SCADA day shows 144 readings per point at the right UTC and stored local stamps that read Pacific civil time, which verifies the UTC-plus-offset workaround (finding 49).
- The nine read types and the ten handheld routes (192 meters, 204 registers) are entered with their starting readings, Cedar Row C as its installer keyed it, and `handheld_configuration` passes all 213 checks.
- The 3 baseline points carry their algorithm, display flag and measured point, each measured point bound to a gateway.

Run `checks.sql` beside this chapter against the instance database to see each of these as a result set. Most read empty until the corresponding entry step has happened; an error, not an empty set, is the sign something is wrong.

Findings

- **HMR_ReadType had no editor anywhere in the product (finding 19).** The table is empty in the installed database; whether a clean install ships rows is still to verify against the factory backup. The shipped `HandheldMeterReadingController` exposes only a GET for it, yet Read Type is a required field on the Handheld Meter Reading Configuration screen. The new Handheld Read Types page (`HMR_ReadType_SaveBatch`, delete only while no register uses the type) is the editor, so Step 5 enters the three Northlake formats there; the catalog script Stage 3 used before was retired on November 5. It closes once the page ships and Northlake's three rows are saved through it.

- **Configuration Tool cannot create a gateway without a type, and no handheld type ships (finding 78).** Add Gateway always assigns a type and the picker cannot clear it, yet the Neptune and MVRS components never read one. Massport's live MVRS gateway has no type and no nodes, which the 7.0 editor cannot reproduce. Northlake uses one empty Handheld Meter Reading Gateway type for its three handheld gateways.
- **The web schedule form cannot leave a Boolean blank (finding 79).** Its checkbox saves true or false, so the MVRS schedule stores UseReadingTimeStamp as false instead of inheriting MVRSGateway:UseReadingTimeStamp. False matches today's default, and 03-workflows-and-tasks.yaml records it as the schedule's value.
- **Several gateway types have no packaged field labels (finding 21).** NOAA and Aeris Weather have no matching row in `GWVendor`, so their nodes and points show no labeled parameters until someone defines them in Gateway Types; Neptune and MVRS are configured on the handheld screens instead. AcquiSuite, BACnet, Modbus, Spinwave, FIG and MV90 shipped labeled, though FIG lacks the Channel, UOM, Multiplier and OffsetTimeMinutes labels this stage adds. Whether the newer types should ship presets is still to confirm.
- **Baseline For waits for a gateway.** BaselinePoints_Lookup lists only measured points that have a gateway or a registered aggregate; with no candidate, saving the Baseline group resets it to [None] without a message.
- **OdbcFileDataGateway is a misnomer inherited from the legacy product.** It is a database-to-file exporter (`SqlExecute`, `If`, `WriteFile`), not a data-acquisition gateway, despite living in the same workflow numbering as the other thirteen. Nothing in this stage exercises it; a future stage with an outbound feed requirement might.
- **KSUU's wind arrives in knots under a name that means km/h (finding 45).** The NOAA gateway keeps the legacy assignment of `wspdm` to the observation's `wind_kt`, and nothing on screen says the value is knots. Northlake sets its KSUU point up in knots; a point built from the field name alone would read 1.852 times too low.
- **A source on standard time all year can only be read through UTC plus an offset (findings 48 and 49).** EEM's time zone list has no fixed-offset zone. The GMT row is no substitute: zone 23 carries the Windows name GMT Standard Time, which is London with summer time (finding 48), and the FIG gateway accepts only `utc` or `local` as its Timestamp Type, although the shared converter already has a standard-time mode that would need no offset (finding 49). So Ridgeline's SCADA is read with the points on Pacific, the task on `utc` and each point's `OffsetTimeMinutes` at 480; the first Ridgeline run verifies that the stored UTC and the stored Pacific local stamps both come out right.
- **No gateway says whether a file stamps interval starts or ends (finding 73).** EEM reads every stamp as an interval end. FIG alone can move a stamp (its per-point offset), MDEF assumes MV90's end stamps, and AcquiSuite and the rest cannot, so the start-stamped AcquiSuite day lands one interval early. Each source format's timestamp contract has to be established before any shift is designed; polls, registers and weather observations do not all want the same one.
- **The wind goals start from what this stage stores (findings 41 and 43).** Possible kW arrives as the turbine controller's own figure; nothing in EEM calculates an expected output or a performance ratio from it, and the scatter plot fits only a straight line. Stage 3 lands and checks the data and leaves the goals to the product.

These are recorded in product findings. If a screen refuses any other value in this stage, add it here with the screen, the field and what happened.